

So, judging from my research, the 'room' in game maker is completely arbitrary and can be ignored; The coordinate grid goes on forever (or rather, until you run into overflow errors, so forever for our purposes) so we can ignore the room size.

Now, for the bad news. We cannot create an infinite map without eventually running into optimization problems. This is because if we keep objects loaded in we will eventually run out of computer resources. With this in mind, I have come up with three solutions:

1. Ignore the problem (the 'nothing' solution). - Ignoring this problem means we don't have to come up with a complex or janky solution for it. This frees up way more time for other aspects of the game, and we don't know how many resources this will eat up, so it *may* be fine to simply ignore it.
2. Make 'factory rooms' (the 'janky' solution). - I envision this as basically making our game on two sets of coordinate grids: The regular grid that we've already been designing with (place down mines, warehouses, etc. on this grid) and a 'meta grid' for optimization where every 'room' is a coordinate on this grid (so, let's say we make each room a regular grid with 32x32 coordinates, that means you can place down whatever you want within the room, and once you have grown to the point of needing more space, you can move to a different room (transfer to a different meta grid coordinate). The camera would be fixed to the center of the meta grid coordinate only 'moving' to the center of the next meta grid coordinate when you change rooms (in reality the camera stays where it is, we just un-render the previous room and render in the new room). You can transfer resources between rooms using conveyor belts.
3. Make 'chunks' - like from Minecraft (the 'complex' solution). - This will be the most difficult solution to implement, but will also be the most in-line with our original concept while allowing for heavy optimization. Basically, we keep the regular grid and meta grid from the factory rooms implementation, but allow for multiple rooms to be open at once and allow for a free-flying camera. This allows us to un-render chunks for optimization (chunks in this case are the rooms/meta grid coordinates from the factory rooms solution). Zooming out would require more loaded rooms, while zooming in would allow for less loaded rooms. We could make the furthest zoom level (and a mini-map if we want to implement it) allow the player to view the map from the meta-grid perspective.

So, what would these meta grid coordinates/rooms entail and why would they increase optimization? Basically, the meta grid is a 2 dimensional dynamic array which stores 2 dimensional static arrays. Each 'coordinate' in the meta grid corresponds to a set of coordinates on the world map, which is how we keep the stuff already placed down saved.

While the player places down entities on tiles, that gets saved to the corresponding coordinate in the chunk's grid in the meta grid. When we un-render a chunk, we delete that chunk from the visible and interactable area and then when we re-render the chunk, we look at the chunk's information from the meta grid and place down exactly what is saved in the chunk's information in the meta grid.

What this means for optimization is no moving sprites in non-rendered chunks, no entities doing anything in non-rendered chunks, but we keep timers to keep resources feeding into the global warehouse inventory as if the chunk was loaded, and generating sprites at the correct rate from the edge of the map if their source is in a non-rendered chunk.